Technical Challenge — TCG Engineering Hub (MVP SaaS)

Stack: Laravel (API) + Vue 3 (UI)

Goal: Build an MVP SaaS used internally by The Cloud Group to execute the TCG Engineering Framework across projects.

0) What we want to see

We are not evaluating design. We are evaluating:

- domain modeling
- API-first architecture
- business rules (gates)
- authorization (RBAC)
- auditability
- clean code + minimal tests

---

1) Core Concepts (Framework → Software)

The system manages Projects.

Each Project contains:

- Artifacts (framework documents/steps)
- Modules (the engineered modules we will build later)

The framework artifacts we want to manage in the MVP are:

1. Strategic Alignment
2. Big Picture

3. Domain Breakdown
4. Module Matrix
5. Module Engineering
6. System Architecture
7. Phase Scope

---

2) MVP Features (Must Have)

A) Authentication + Roles (RBAC)

- Login (Sanctum recommended)
- Roles: admin, pm, engineer, viewer
- Permissions minimum:
  - admin: full access
  - pm: manage projects, artifacts, modules
  - engineer: edit modules, view artifacts
  - viewer: read-only

B) Projects (CRUD)

Entity: Project

- name
- client_name (string)
- status: draft | discovery | execution | delivered
- created_by

Required endpoints:

- list/create/show/update/archive (soft delete ok)

UI:

- project list
- project detail

C) Artifacts (CRUD + Gates)

Entity: Artifact

- project_id
- type (enum):

  strategic_alignment | big_picture | domain_breakdown | module_matrix | module_engineering | system_architecture | phase_scope

- status: not_started | in_progress | blocked | done
- owner_user_id (nullable)
- content_json (json) — structured fields
- completed_at (nullable)

Artifact minimal fields (content_json)

You can implement them as JSON schema-like structure.

Strategic Alignment

- transformation (text)
- supported_decisions (array of strings)
- measurable_success (array of {metric, target})
- out_of_scope (array of strings)

Big Picture

- ecosystem_vision (text)
- impacted_domains (array of strings)

- success_definition (text)

Domain Breakdown

- domains (array of {name, objective, owner_user_id})

Module Matrix

- modules_overview (array of {name, domain, priority, phase})

System Architecture

- auth_model (text)
- api_style (text)
- data_model_notes (text)
- scalability_notes (text)

Phase Scope

- included_modules (array of module_ids)
- excluded_items (array of strings)
- acceptance_criteria (array of strings)

Gates (Business Rules) — mandatory

Implement these rules:

Gate 1: You cannot mark domain_breakdown as done if big_picture is not done.

Gate 2: You cannot mark module_matrix as done if domain_breakdown is not done.

Gate 3: You cannot mark system_architecture as done unless there are at least N validated modules (N configurable, default 3).

Gate 4: You cannot move Project status from discovery → execution unless:

- strategic_alignment is done
- big_picture is done
- domain_breakdown is done
- module_matrix is done

These rules must live in a domain service (not controller).

UI must show: why it's blocked.

D) Modules (CRUD with TCG fields)

Entity: Module

- project_id
- domain (string or FK to Domain table—your choice)
- name
- status: draft | validated | ready_for_build

Module fields (THE IMPORTANT PART)

Create these fields exactly (can be columns or JSON, but must be queryable where it matters):

1. objective (text)
2. inputs (json array)
3. data_structure (text or json)
4. logic_rules (text)
5. outputs (json array)

6. responsibility (text) ← who/what owns this module operationally
7. failure_scenarios (text)
8. audit_trail_requirements (text)
9. dependencies (json array of module_ids)
10. version_note (string) (simple versioning note)

Module validation rule (mandatory)

A module can only move to validated if:

- objective is not empty
- inputs has at least 1 item
- outputs has at least 1 item
- responsibility is not empty

UI:

- module list within project
- module detail editor with these fields
- button: “Validate module” (disabled if rules not satisfied)

E) Audit Trail (mandatory)

Entity: AuditEvent

- actor_user_id
- entity_type (project | artifact | module)
- entity_id
- action (created | updated | status_changed | validated | completed)
- before_json
- after_json
- created_at

Log at least:

- project status changes
- artifact status changes

- module status changes (draft→validated etc.)
- edits to artifacts/modules

UI:

- project timeline showing audit events

---

3) API Requirements

- /api/v1/...
- Consistent response format (success + error)
- Validation errors structured
- Pagination for lists (projects/modules/audit)

---

4) Frontend Requirements (Vue 3)

Must have pages:

1. Login
2. Projects list
3. Project detail:
    - Artifacts list (status, owner, blocked reason)
    - Modules list
    - Audit timeline
4. Artifact detail edit
5. Module detail edit

Minimum UX:

- loading states
- error states
- hide actions if no permission
- show blocking reasons for gates

---

5) Tests (minimum)

Laravel feature tests (at least 4):

1. Gate 1 enforcement (domain_breakdown cannot be done if big_picture not done)
2. Module validation rule (cannot validate if missing required fields)
3. Project cannot move discovery→execution if missing required artifact statuses
4. Authorization: viewer cannot edit modules/artifacts

---

6) Deliverables

- Repo with backend + frontend (monorepo or two repos)
- README:
  - setup
  - seed demo users (admin/pm/engineer/viewer)
  - run tests
- Short notes (max 1 page):
  - architecture decisions
  - what you'd improve next (templates, exports, etc.)

---

7) Evaluation (how we score)

100 points

- Data model & clarity (20)
- Gates implemented cleanly (20)
- Modules implemented with all framework fields (20)
- RBAC correct (15)
- Audit trail useful (15)
- Tests + code quality (10)

Red flags:

- Rules in controllers
- No audit diffs
- No permission checks
- "Validate" without enforcing rules server-side

Opcionales (solo si te da tiempo)

- “Templates”: ability to create a new project with pre-created artifacts.
- Simple “export” to JSON for a